

# PhishGuard

## Email Phishing Detection Model

**Complete Training Guide:** datasets, features, step-by-step training, expected results

| Item             | Value                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository       | <a href="https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool">https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool</a> |
| Branch           | claude/blissful-dijkstra-fheyhs                                                                                                                 |
| Code verified at | commit 71ff4ea (Downloader: retry transient corruption)                                                                                         |
| Model            | TF-IDF (word 1-2 grams) + Logistic Regression, scikit-learn 1.9.1                                                                               |
| Training data    | 6,125 phishing + 30,304 legitimate emails (+ 56 curated)                                                                                        |
| Guide date       | 7 October 2026                                                                                                                                  |

**How this guide was verified.** Every command in sections 4-9 and 14 was executed on a **fresh clone** of the branch above (Linux, Python 3.13): install, download, hash checks, training, tests and the post-training commands. The freshly trained model produced metrics **identical** to the ones printed in section 13. The test suite (210 tests) also passes on Python 3.11. Windows and macOS commands are given but were **not** executed by the author.

# Contents

1. What you are training
2. Requirements
3. Safety rules (read first)
4. Quick start (one command)
5. Step-by-step training
6. Dataset catalogue and links
7. Integrity pins (hashes and commit)
8. Manual download (if the script cannot run)
9. How the data is prepared
10. Features
11. Model and training settings
12. How the model is evaluated
13. Expected results
14. Using and inspecting your model
15. Adding your own examples
16. Troubleshooting
17. Metric glossary
18. Known limitations
19. Licences and attribution

# 1. What you are training

PhishGuard judges an email with three independent layers. **Only the text model is trained**; the other two are deterministic rules that need no training. They will be combined with the text model in the next project stage (Day 6 scorer).

| Layer                   | Module           | Type                       | What it judges                                                                                  |
|-------------------------|------------------|----------------------------|-------------------------------------------------------------------------------------------------|
| Text model (this guide) | text_model.py    | Machine learning (trained) | Phishing intent in subject + body: urgency, requests to verify, pay, transfer, reply with codes |
| Header checks           | header_checks.py | Rules (18 checks)          | SPF/DKIM/DMARC results, spoofed display names, Reply-To tricks, lookalike sender domains        |
| URL & attachment checks | url_checks.py    | Rules (27 checks)          | Lookalike / punycode / IP links, link-text mismatch, embedded forms, risky attachments          |

Why the text model ignores links and sender addresses: they are judged by the rule layers, and leaving them in the text taught an early model the shortcut "has a link = phishing" (see section 9).

## Pipeline

```
raw .eml / CSV row
-> safe parser (size limits, no HTML rendering)
-> subject + visible body text
-> normalise (drop URLs/emails, numbers->numtoken)
-> TF-IDF word 1-2 grams (corpus artefacts removed)
-> Logistic Regression -> phishing probability 0..1
-> top_terms(): words that pushed the score up

parser.py
email_text()
normalize_text()
TfidfVectorizer
LogisticRegression
TextModel
```

# 2. Requirements

| Requirement      | Minimum                                               | Notes                                                                                                                                    |
|------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Operating system | Linux, macOS or Windows 10/11                         | Tested on Linux. On Windows, WSL2 or a VM is recommended (see 3).                                                                        |
| Python           | 3.11 or newer                                         | Tested with 3.11.17 and 3.13.                                                                                                            |
| git              | any recent version                                    | Required: the honeypot dataset is fetched with git.                                                                                      |
| Free disk space  | 4 GB                                                  | Measured: data/raw 2.6 GB, data/processed 65 MB, .venv 428 MB.                                                                           |
| RAM              | 2 GB free                                             | Measured peak during training: 1.13 GB.                                                                                                  |
| Internet access  | github.com,<br>raw.githubusercontent.com,<br>pypi.org | HTTPS only.                                                                                                                              |
| Time             | about 7 minutes (first run)                           | Measured end-to-end (clone + install + download + parse + train): 6 min 24 s, of which install 26 s. Retraining later: about 2 min 40 s. |

# 3. Safety rules (read first)

The training data contains **real phishing emails**, and the honeypot set contains **real malicious attachments**. Handled as described here they are harmless, because PhishGuard only reads them as text.

- **Never open** any file in data/raw/ with a mail client, browser, Word or a PDF reader. Never click links in them.
- PhishGuard parses them **statically** with its hardened parser (12,269 real honeypot emails parsed with 0 crashes).
- Work in a virtual environment; preferably inside a VM or WSL2 so the samples stay off your daily-use system.
- **Antivirus**: it may quarantine some honeypot files. That is expected and safe; do **not** add exclusions. Your sample count, and therefore your metrics, may then differ slightly from section 13.
- data/raw/ and data/processed/ are gitignored. Never commit or share them.
- The honeypot set is licensed **CC BY-NC 4.0**: non-commercial use only.
- Never print or log full email bodies (the code never does).

## 4. Quick start (one command)

### Linux / macOS

```
git clone -b claud/blissful-dijkstra-fheyhs \
  https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python scripts/bootstrap.py
```

### Windows (PowerShell)

```
git clone -b claud/blissful-dijkstra-fheyhs `
  https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e ".[dev]"
python scripts/bootstrap.py
```

bootstrap.py downloads and verifies all datasets, then trains, but only if no valid model exists. Re-running it is safe; it prints [ok] email\_model present ... nothing to do. Use python scripts/bootstrap.py --force to retrain anyway. If PowerShell blocks Activate.ps1, run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once.

## 5. Step-by-step training

Use this instead of the quick start if you want to see and check each stage.

### Step 1 - Get the code

```
git clone -b claud/blissful-dijkstra-fheyhs \
  https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
```

In Windows PowerShell, end the first line with a backtick (`) instead of a backslash, as in section 4.

### Step 2 - Create a virtual environment and install

```
python3 -m venv .venv          # Windows: python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

This installs PhishGuard plus pandas, scikit-learn, tldextract, joblib, pytest, ruff, bandit and pip-audit.

### Step 3 - Download and verify the datasets

```
python scripts/download_data.py
```

Expected output (first run). Every line must be [done] or [ok]:

```
[done] Nazario.csv: 7.8 MB
[done] Nigerian_Fraud.csv: 9.2 MB
[done] Enron.csv: 45.6 MB
[done] SpamAssasin.csv: 14.9 MB
[done] CEAS_08.csv: 67.9 MB
[done] Ling.csv: 9.3 MB
[done] sms.tsv: 0.5 MB
[done] phishing_pot at pinned commit 89e2bc05d159
```

A [retry] line means a transfer was corrupted in transit and is being re-downloaded (the author saw this happen once). A file is accepted **only** if its SHA-256 matches the pin. A [fail] line after 3 attempts means: do not train; see section 16.

### Step 4 - Train

```
python -m phishguard.text_model train-email
```

The first run parses about 12,000 honeypot emails (about 2 minutes) and caches the extracted text in `data/processed/`. Then it trains the final model plus four leave-one-corpus-out probe models. The first output line must be exactly (shown wrapped here; it is one line):

```
corpus after de-duplication: {'CEAS_08.csv:0': 16968, 'Enron.csv:0': 14444,
'Ling.csv:0': 2396, 'Nazario.csv:1': 1526, 'Nigerian_Fraud.csv:1': 3200,
'SpamAssasin.csv:0': 4073, 'phishing_pot:1': 3349}
```

The last line must be saved `models/email_model.joblib` (sha256 recorded in `models/manifest.json`).

## Step 5 - Verify

```
pytest # expected: 210 passed
python scripts/bootstrap.py # expected: [ok] email_model present ... nothing to do
```

Quality tests check your trained model against fixed bars (e.g. recall at least 0.95, future-phishing recall at least 0.85). If they pass, your model meets the documented quality.

## Step 6 - Optional: ablation study (about 2 minutes)

```
python scripts/ablation.py
```

Trains three models on different data and scores them on identical tests (results in section 13.5).

## Files produced

| Path                                              | Content                                                    | In git?         |
|---------------------------------------------------|------------------------------------------------------------|-----------------|
| <code>data/raw/</code>                            | Downloaded datasets (about 2.6 GB)                         | No (gitignored) |
| <code>data/processed/phishing_pot_v2_*.csv</code> | Cached normalised honeypot text                            | No (gitignored) |
| <code>models/email_model.joblib</code>            | The trained model (a pickle)                               | No (gitignored) |
| <code>models/manifest.json</code>                 | Model SHA-256, training time, sklearn version, all metrics | Yes             |

**Security note:** `.joblib` files are pickles, and loading a pickle can execute code. `load_model()` refuses to load a model unless its SHA-256 matches `models/manifest.json`. Never load a model file someone sends you.

## 6. Dataset catalogue and links

All files are downloaded automatically by `scripts/download_data.py`. Labels in the raw CSVs: 1 = phishing/spam, 0 = legitimate. PhishGuard uses **only** the rows listed under "Rows used" (spam rows are excluded because spam is not phishing).

| Dataset                    | Raw rows (label 0 / 1)       | Rows used (after de-dup) | Columns (raw features)                             | Our label  |
|----------------------------|------------------------------|--------------------------|----------------------------------------------------|------------|
| Nazario.csv                | 1,565 (0 / 1,565)            | 1,526: all               | sender, receiver, date, subject, body, urls, label | phishing   |
| Nigerian_Fraud.csv         | 3,332 (0 / 3,332)            | 3,200: all               | sender, receiver, date, subject, body, urls, label | phishing   |
| phishing_pot (honeypot)    | 12,271 .eml files            | 3,349: English only      | full raw email (.eml): headers, body, attachments  | phishing   |
| Enron.csv                  | 29,767 (15,791 / 13,976)     | 14,444: label 0          | subject, body, label                               | legitimate |
| SpamAssasin.csv            | 5,809 (4,091 / 1,718)        | 4,073: label 0           | sender, receiver, date, subject, body, label, urls | legitimate |
| CEAS_08.csv                | 39,154 (17,312 / 21,842)     | 16,968: label 0          | sender, receiver, date, subject, body, label, urls | legitimate |
| Ling.csv                   | 2,859 (2,401 / 458)          | 2,396: label 0           | subject, body, label                               | legitimate |
| modern_lures.csv (in repo) | 112 (56 / 56)                | 56 train + 56 test       | id, split, label, category, text                   | both       |
| sms.tsv                    | 5,572 (ham 4,825 / spam 747) | not used yet (Day 6)     | label (ham/spam), text (tab-separated)             | SMS        |

Only the **subject** and **body** columns (or the parsed subject + visible body of an .eml) are used by the text model.

## Download links (exactly what the script fetches)

```
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Nazario.csv
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Nigerian_Fraud.csv
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Enron.csv
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/SpamAssasin.csv
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/CEAS_08.csv
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Ling.csv
https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv
https://github.com/rf-peixoto/phishing_pot.git (folder email/ only)
```

## Original sources

| Dataset              | Original source / citation                                                                            | Link                                                                                                          |
|----------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Email CSVs (all six) | Per-source files of the Kaggle "Phishing Email Dataset" (Al-Subaiey et al., 2024), mirrored on GitHub | kaggle.com/datasets/naserabdullahalam/phishing-email-dataset<br>github.com/rokibulroni/Phishing-Email-Dataset |
| Nazario              | Jose Nazario phishing corpus                                                                          | via the compilation above                                                                                     |
| Nigerian_Fraud       | CLAIR collection of fraud email (Radev)                                                               | via the compilation above                                                                                     |
| Enron                | Enron-Spam corpus (Metsis et al., 2006)                                                               | via the compilation above                                                                                     |
| SpamAssassin         | Apache SpamAssassin public corpus                                                                     | via the compilation above                                                                                     |
| CEAS_08              | CEAS 2008 Live Spam Challenge corpus                                                                  | via the compilation above                                                                                     |
| Ling                 | Ling-Spam corpus (Androustopoulos et al., 2000)                                                       | via the compilation above                                                                                     |
| phishing_pot         | Real phishing caught by honeypots, 2022-2026 (R. Peixoto)                                             | github.com/rf-peixoto/phishing_pot                                                                            |
| sms.tsv              | UCI SMS Spam Collection (Almeida & Hidalgo, 2011)                                                     | archive.ics.uci.edu/dataset/228/sms+spam+collection                                                           |

The GitHub links were accessed and downloaded by the author. The Kaggle and UCI links come from search results; those sites were blocked in the author's environment, so they were not opened. The original hosts (Kaggle, UCI, monkey.org, CMU) being blocked is also why public GitHub mirrors are used. The mirrors are third-party copies; the pins in section 7 guarantee that you get the exact same bytes as the author, not that the mirror is authentic.

## 7. Integrity pins (hashes and commit)

These values are hard-coded in scripts/download\_data.py. The script refuses any file that does not match.

```
Nazario.csv          b8fbc4158fbdaff1ed98584c43d72e283c6352d7ade4d457d34c1d79488d184
Nigerian_Fraud.csv   f5d6930763cae6feeae6484131c79655a7440abb97d6f51eab757aea4e6ab4b
Enron.csv            99933d3233510cf5dc2ee7768ddc609425cb8002bf5d7691d9a7157ffa5fd318
SpamAssasin.csv      3bfe8f8abff89f69a98456be50413c2fcb20a476141116dd84ee1507b980c00e
CEAS_08.csv          22375e7d5f5a8229dbe987914ee9b3705656c590038662a7df6054629b376074
Ling.csv             c133792260f18b251e9377b9cb31bef226322af6dc0d79841f61c67489929eca
sms.tsv              7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d

phishing_pot         git commit 89e2bc05d15955389782f2f8e8d916588cd49cd
```

Additional download protections: HTTPS only (the downloader has no http:, file: or ftp: handler, so even a redirect cannot downgrade), 120 MB size cap per file, truncation check against Content-Length, up to 3 attempts, atomic writes (a partial file never looks valid), and git transport restricted to HTTPS.

## 8. Manual download (if the script cannot run)

Only needed if download\_data.py cannot run on your machine. The files must end up exactly here:

```
data/raw/Nazario.csv   data/raw/Nigerian_Fraud.csv  data/raw/Enron.csv
data/raw/SpamAssasin.csv data/raw/CEAS_08.csv        data/raw/Ling.csv
data/raw/sms.tsv       data/raw/phishing_pot/email/sample-*.eml
```

## Linux / macOS

```
mkdir -p data/raw && cd data/raw
M=https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main
for f in Nazario.csv Nigerian_Fraud.csv Enron.csv SpamAssasin.csv CEAS_08.csv Ling.csv; do
  curl -fsSL -o "$f" "$M/$f"
done
curl -fsSL -o sms.tsv \
  https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv
sha256sum *.csv sms.tsv      # macOS: shasum -a 256 *.csv sms.tsv
```

Compare every hash with section 7. Example of an automatic check for one file:

```
echo "7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d  sms.tsv" | sha256sum -c -
```

Honeypot (run inside data/raw):

```
git init phishing_pot && cd phishing_pot
git remote add origin https://github.com/rf-peixoto/phishing_pot.git
git sparse-checkout set --no-cone email/
git fetch --depth 1 --filter=blob:none origin 89e2bc05d159555389782f2fbe8d916588cd49cd
git checkout 89e2bc05d159555389782f2fbe8d916588cd49cd
git rev-parse HEAD      # must print the commit above
```

## Windows (PowerShell)

```
New-Item -ItemType Directory -Force data\raw | Out-Null; Set-Location data\raw
$M = "https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main"
foreach ($f in "Nazario.csv","Nigerian_Fraud.csv","Enron.csv","SpamAssasin.csv",
  "CEAS_08.csv","Ling.csv") { Invoke-WebRequest "$M/$f" -OutFile $f }
$S = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
Invoke-WebRequest $S -OutFile sms.tsv
Get-FileHash *.csv, sms.tsv -Algorithm SHA256
```

The honeypot git commands are the same in PowerShell. Afterwards run python scripts/download\_data.py once: it should report every file as [ok] ... verified.

## 9. How the data is prepared

All of this happens automatically inside `load_email_corpus()` and `load_honeypot()`.

- **Label filtering:** phishing = all Nazario and Nigerian\_Fraud rows + English honeypot emails; legitimate = label-0 rows of Enron, SpamAssassin, CEAS\_08 and Ling. Spam rows are dropped.
- **Text used:** subject + newline + body (for .eml files: parsed subject + visible text from plain-text and HTML parts; scripts and styles dropped; attachments never read as text).
- **Normalisation** (identical at training and prediction time): text cut to 40,000 characters; URLs (http://, https://, www.) and email addresses **removed**; every number replaced by numtoken; whitespace collapsed; lower-cased; capped at 20,000 characters.
- **Honeypot placeholder:** the corpus replaces victims' addresses with phishing@pot; it is removed before normalisation.
- **English filter (honeypot only):** a message is kept if it contains at least 5 of these 32 English function words and they make up at least 8% of its tokens: all, an, and, are, as, at, be, by, can, for, from, has, have, if, in, is, it, not, of, on, or, our, please, that, the, this, to, we, will, with, you, your. Only about 43% of the honeypot is English (estimated from a 1-in-5 sample); using all of it would teach the model "non-English = phishing".
- **Short texts:** rows with fewer than 20 characters after normalisation are dropped.
- **De-duplication:** rows whose first 2,000 normalised characters are identical are removed; the first copy wins, so a text can never appear twice, with the same or with conflicting labels.
- **Curated examples:** data/curated/modern\_lures.csv (112 hand-written modern messages, 27 categories). The train half is added with sample weight 20; the test half is only used for evaluation.

## 10. Features

### 10.1 Text model features (what the trained model sees)

| Setting         | Value                    | Meaning                                                                            |
|-----------------|--------------------------|------------------------------------------------------------------------------------|
| Feature type    | TF-IDF word n-grams      | Each word and word pair gets a weight: frequent in this email, rare overall = high |
| ngram_range     | (1, 2)                   | Single words and two-word phrases, e.g. your account, click here                   |
| token_pattern   | (?u)\b[a-z][a-z]+\b      | Words of 2+ letters (numbers are already numtoken)                                 |
| min_df / max_df | 3 / 0.9                  | Ignore terms in fewer than 3 emails or in more than 90% of emails                  |
| max_features    | 50,000                   | Vocabulary size limit                                                              |
| sublinear_tf    | True                     | Uses $1 + \log(\text{term count})$ , so repetition is not over-rewarded            |
| strip_accents   | unicode                  | é becomes e, etc.                                                                  |
| stop_words      | 51 corpus-artefact words | Removed because they identify a dataset, not phishing (list below)                 |

**Stop words (corpus artefacts):** enron, vince, kaminski, louise, daren, houston, ect, hou, jeff, monkey, jose, nazario, utf, iso, charset, http, https, www, com, net, org, html, htm, nbsp, subject, cc, fw, fwd, url, pot, phishing, unsubscribe, preferences, copyright, reserved, app, apps, ai, de, para, que, da, em, la, el, und, der, die, enviado, assunto, remetente

Why these words: enron, vince, kaminski, louise, daren, houston, ect, hou, jeff are Enron names/places; monkey, jose, nazario come from the Nazario mailbox; utf, iso, charset, nbsp, html, htm are encoding residue; http, https, www, com, net, org, url are leftovers of broken-up URLs; pot, phishing come from the honeypot placeholder (and "phishing" never appears in the legitimate corpora); unsubscribe, preferences, copyright, reserved, app, apps, ai only appear in modern mail, which in this data is always phishing (era shortcut); the rest are Portuguese/Spanish/German residue (language shortcut). Each was found by inspecting the model's top-weighted words.

### 10.2 Header rule features (not trained; used by the scorer)

| Feature (code)           | Weight | Fires when                                                                                        |
|--------------------------|--------|---------------------------------------------------------------------------------------------------|
| dmarc_fail               | 0.35   | DMARC result is fail (sender domain did not authorise this message)                               |
| spf_fail                 | 0.20   | SPF result is fail or permerror                                                                   |
| spf_softfail             | 0.10   | SPF result is softfail                                                                            |
| dkim_fail                | 0.15   | DKIM signature invalid (fail or permerror)                                                        |
| forged_auth_header       | 0.30   | A lower Authentication-Results header from the trusted server claims a pass the top header denies |
| auth_unverified          | 0.05   | No Authentication-Results header, or the top one is not from TRUSTED_AUTHSERV_ID                  |
| reply_to_mismatch        | 0.25   | Reply-To domain differs from the From domain                                                      |
| reply_to_freemail        | 0.10   | Replies redirected to a free-mail domain while the sender is not free mail                        |
| return_path_mismatch     | 0.05   | Bounce (Return-Path) domain differs from From (skipped for official brand senders)                |
| display_name_brand_spoof | 0.35   | Display name names one of 51 brands but the address is not on that brand's official domains       |
| display_name_other_email | 0.30   | Display name contains a different email address than the real sender                              |
| freemail_impersonation   | 0.25   | Free-mail sender whose display name claims a role (CEO, HR, payroll, bank, police, ...)           |
| lookalike_sender_domain  | 0.40   | Sender domain imitates a brand: homoglyphs, digit swaps, typos, brand + extra words               |
| punycod_sender_domain    | 0.20   | Sender domain contains an IDN (xn--) label                                                        |
| bulk_recipients          | 0.05   | 10 or more visible To + Cc recipients                                                             |
| undisclosed_recipients   | 0.05   | No visible recipients (hidden BCC mass mailing)                                                   |
| missing_from             | 0.20   | No valid From address                                                                             |
| missing_message_id       | 0.05   | Message-ID header missing                                                                         |

Header risk = sum of fired weights, capped at 1.0. SPF/DKIM/DMARC are read from the receiving server's Authentication-Results header (no DNS lookups). Only the top-most header from TRUSTED\_AUTHSERV\_ID (default mx.google.com) may credit a pass; failures always count.

## 10.3 URL and attachment rule features (not trained; used by the scorer)

| Feature (code)           | Weight | Fires when                                                                          |
|--------------------------|--------|-------------------------------------------------------------------------------------|
| url_dangerous_scheme     | 0.40   | javascript:, data:, vbscript: or file: link                                         |
| url_ip_host              | 0.30   | Link host is an IP address                                                          |
| url_obfuscated_ip        | 0.40   | Decimal, hex or octal IP host (e.g. 3232235777, 0x7f.1)                             |
| url_at_symbol            | 0.35   | user@host trick: https://paypal.com@evil.test goes to evil.test                     |
| url_obfuscated_host      | 0.25   | Backslash, %-encoding or invisible/bidi characters in the URL                       |
| url_punycode             | 0.30   | IDN host (xn-- or raw Unicode such as Cyrillic look-alikes)                         |
| url_lookalike_domain     | 0.40   | Domain imitates a brand (homoglyph, typo, brand + words)                            |
| url_brand_outside_domain | 0.30   | Brand named in the subdomain or path of someone else's site                         |
| url_shortener            | 0.10   | Link shortener (19 known, e.g. bit.ly); weak signal                                 |
| url_free_hosting         | 0.15   | Free site builder / tunnel (22 known, e.g. netlify.app, ngrok)                      |
| url_cloud_user_content   | 0.10   | User-uploaded cloud content (S3, GCS, Firebase, Azure Blob/Web, Google Forms, IPFS) |
| url_risky_tld            | 0.10   | High-abuse TLD (22 listed, e.g. .zip .top .xyz)                                     |
| url_http_only            | 0.05   | Explicit unencrypted http://                                                        |
| url_many_subdomains      | 0.10   | 4 or more labels before the registered domain                                       |
| url_nonstandard_port     | 0.10   | Port other than 80/443 (or invalid)                                                 |
| url_credential_words     | 0.10   | login/verify/kyc/password... in path or query (not on the brand's own domain)       |
| url_long                 | 0.05   | URL longer than 100 characters                                                      |
| url_malformed            | 0.10   | URL cannot be parsed                                                                |
| link_text_mismatch       | 0.45   | Visible link text shows one domain, the link goes to another (strongest signal)     |
| link_text_brand_mismatch | 0.25   | Link text names a brand, the link goes elsewhere                                    |
| form_in_email            | 0.35   | HTML form inside the email (credential harvesting)                                  |
| att_executable           | 0.50   | Executable / script / disk-image attachment (24 extensions)                         |
| att_html                 | 0.35   | HTML or SVG attachment (local phishing page)                                        |
| att_macro                | 0.35   | Macro-enabled Office file (.docm, .xlsm, ...)                                       |
| att_double_extension     | 0.30   | Double extension such as invoice.pdf.exe                                            |
| att_rtlo_filename        | 0.40   | Right-to-left-override character disguising the extension                           |
| att_archive              | 0.15   | Archive attachment (.zip, .rar, .7z, ...)                                           |

Link/attachment risk = sum of fired weights (each code counted once per email), capped at 1.0. Links are never fetched or resolved (no SSRF, no tipping off the attacker).

## 11. Model and training settings

| Setting                    | Value                              | Why                                                             |
|----------------------------|------------------------------------|-----------------------------------------------------------------|
| Algorithm                  | Logistic Regression                | Fast, robust, and every word has a visible weight (explainable) |
| C (inverse regularisation) | 4.0                                | Moderate regularisation                                         |
| class_weight               | balanced                           | Phishing is the minority class (about 1 in 6 training rows)     |
| solver / max_iter          | liblinear / 1000                   | Deterministic solver                                            |
| random_state / seed        | 42                                 | Reproducible splits and training                                |
| Curated sample weight      | 20                                 | Fixed in advance (not tuned on the test set)                    |
| Train / test split         | 80% / 20% stratified               | For the six CSV sources                                         |
| Honeypot split             | oldest 70% train / newest 30% test | By sample number = time order                                   |
| Training rows              | 6,125 phishing / 30,304 legitimate | Plus 56 curated train rows                                      |

| Source      | Training rows |
|-------------|---------------|
| CEAS_08.csv | 13,578        |
| Enron.csv   | 11,517        |

| Source             | Training rows |
|--------------------|---------------|
| Ling.csv           | 1,932         |
| Nazario.csv        | 1,203         |
| Nigerian_Fraud.csv | 2,578         |
| SpamAssasin.csv    | 3,277         |
| phishing_pot       | 2,344         |

## 12. How the model is evaluated

| Test                 | What it measures                                              | Data                                       |
|----------------------|---------------------------------------------------------------|--------------------------------------------|
| In-distribution      | Classic accuracy on unseen emails from the same sources       | Stratified 20% of the six CSV sources      |
| Honeypot future test | Catching <b>next month's</b> real phishing                    | Newest 30% of honeypot emails (time split) |
| Modern test          | Today's lures and alarming-but-genuine notices                | Held-out half of the 112 curated messages  |
| Leave-one-corpus-out | False alarms on legitimate mail in an <b>unfamiliar style</b> | Each legitimate corpus held out in turn    |

All results are stored in `models/manifest.json` every time you train.

## 13. Expected results

A correct run reproduces these numbers **exactly** (if antivirus removed honeypot files, they may differ slightly).

### 13.1 In-distribution test

| Metric              | Value           | Plain English                                 |
|---------------------|-----------------|-----------------------------------------------|
| Emails tested       | 8,522           | 945 phishing, 7,577 legitimate                |
| Accuracy            | 99.47%          | 8,477 of 8,522 correct                        |
| Precision           | 96.88%          | When it says phishing, it is right this often |
| Recall              | 98.41%          | Share of phishing caught                      |
| F1                  | 0.9764          | Balance of precision and recall               |
| False-positive rate | 0.40%           | Share of legitimate emails wrongly flagged    |
| ROC-AUC / PR-AUC    | 0.9997 / 0.9981 | Ranking quality (1.0 = perfect)               |

|                             | Predicted phishing | Predicted legitimate |
|-----------------------------|--------------------|----------------------|
| Actually phishing (945)     | 930 caught         | 15 missed            |
| Actually legitimate (7,577) | 30 false alarms    | 7,547 correct        |

### 13.2 Honeypot future test (real 2026 phishing, never trained on)

| Emails | Recall at 0.5 | Recall at 0.8 (quarantine threshold) |
|--------|---------------|--------------------------------------|
| 1,005  | 87.9%         | 67.4%                                |

### 13.3 Modern test (56 hand-written messages)

| Accuracy | Precision | Recall | F1    | False-positive rate | Errors by category                                                                 |
|----------|-----------|--------|-------|---------------------|------------------------------------------------------------------------------------|
| 87.5%    | 88.9%     | 85.7%  | 0.873 | 10.7%               | bec_giftcard 1, hinglish 2, in_upi 1, security_notice 1, utility 1, vendor_legit 1 |

### 13.4 Leave-one-corpus-out (false alarms on unfamiliar legitimate mail)

| Held-out corpus | Emails | Flagged at 0.5 | Flagged at 0.8 | Phishing recall of probe |
|-----------------|--------|----------------|----------------|--------------------------|
| CEAS_08.csv     | 16,968 | 4.83%          | 2.17%          | 98.94%                   |
| Enron.csv       | 14,444 | 8.64%          | 1.13%          | 99.05%                   |
| Ling.csv        | 2,396  | 0.79%          | 0.17%          | 98.52%                   |
| SpamAssasin.csv | 4,073  | 0.83%          | 0.10%          | 98.62%                   |

## 13.5 Ablation: what each dataset contributes

From python scripts/ablation.py (identical test sets for all three variants):

| Training data                                | Future phishing recall | Modern recall / FPR | Old-source F1 / FPR |
|----------------------------------------------|------------------------|---------------------|---------------------|
| A: Nazario + Nigerian + Enron + SpamAssassin | 40.60%                 | 78.57% / 7.14%      | 0.9867 / 0.21%      |
| B: A + CEAS_08 + Ling legitimate             | 29.95%                 | 78.57% / 7.14%      | 0.9867 / 0.16%      |
| C: B + honeypot (final model)                | 87.86%                 | 85.71% / 10.71%     | 0.9862 / 0.30%      |

**How to read this honestly:** accuracy is flattering because about 89% of test emails are legitimate (a model that always says "legitimate" would already score about 89%). The most realistic figure is the **honeypot future recall**: real attacks newer than anything the model saw. A model trained only on the classic datasets (variant A) caught just 40.6% of them despite about 99% accuracy on its own test split.

## 14. Using and inspecting your model

### View all metrics

```
python -c "  
import json  
m = json.load(open('models/manifest.json'))['email_model']['metrics']  
print(json.dumps(m, indent=2))  
"
```

### Score your own text

```
python -c "  
from phishguard.text_model import load_model, TextModel  
tm = TextModel(load_model('email_model'))  
msg = 'Dear customer, your account will be suspended. Verify your password today.'  
print(round(tm.predict_proba(msg), 3), tm.top_terms(msg, 5))  
"
```

Expected output (verified): 1.0 [('your', 1.926), ('account', 1.34), ('dear', 1.113), ('verify', 0.841), ('dear customer', 0.777)]

### See which words the model relies on

```
python -c "  
from phishguard.text_model import load_model  
import numpy as np  
m = load_model('email_model')  
words = m.named_steps['tfidf'].get_feature_names_out()  
coef = m.named_steps['clf'].coef_[0]  
print('phishing words:', [words[i] for i in np.argsort(coef)[-15:][::-1]])  
print('legit words: ', [words[i] for i in np.argsort(coef)[:15]])  
"
```

Always look at these lists after changing data. Names, dataset-specific words or language words at the top mean the model learned a shortcut (section 9). This is how every artefact in the stop-word list was found.

On Windows PowerShell, put each Python snippet into a file (e.g. check.py) and run python check.py.

## 15. Adding your own examples

To teach the model lures you see (e.g. a new local scam), add rows to data/curated/modern\_lures.csv:

```
id,split,label,category,text  
113,train,1,in_upi,"Your refund is pending. Enter your UPI PIN to receive Rs 2,000."  
114,test,1,in_upi,"Cashback of Rs 500 credited. Approve the collect request to claim."  
115,train,0,otp,"Your OTP is 123456. Never share it with anyone, including bank staff."
```

- id unique; split is train or test; label 1 = phishing, 0 = legitimate.
- Put examples of every category in **both** splits, and add legitimate look-alikes too, or the model learns that the topic itself means phishing.

- Never put real personal data, real links or real addresses in this file (it is committed to git; a test enforces no links or addresses).
- Then retrain: `python -m phishguard.text_model train-email` and run `pytest`.
- Do **not** tune settings until the test-split numbers look good; that silently turns your test set into training data.

## 16. Troubleshooting

| Symptom                                                  | Cause                                         | Fix                                                                                            |
|----------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------|
| [fail] ... SHA-256 mismatch ... gave up after 3 attempts | Corrupted transfers, or the mirror changed    | Run the script again. If it persists, do not train: the data is not what this guide documents. |
| [fail] ... 403 or connection errors                      | Network/firewall blocks GitHub                | Allow github.com and raw.githubusercontent.com, or use another network.                        |
| git is required for git-hosted datasets                  | git not installed or not on PATH              | Install git (git-scm.com) and open a new terminal.                                             |
| FileNotFoundError ... run scripts/download_data.py       | Training started before the download finished | Run the download first; every line must be [done] or [ok].                                     |
| ModelIntegrityError ... does not match its manifest hash | Model file changed or came from elsewhere     | Run <code>python scripts/bootstrap.py</code> (retrains). Never bypass this check.              |
| Fewer than 3,349 phishing_pot rows                       | Antivirus removed honeypot files              | Expected; metrics differ slightly. Don't add AV exclusions; a VM/WSL2 avoids it.               |
| python: command not found / wrong version                | Python missing or older than 3.11             | Install Python 3.11+; on Linux/macOS use <code>python3</code> .                                |
| Activate.ps1 cannot be loaded                            | PowerShell execution policy                   | <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>                               |
| Quality tests fail after your changes                    | Your data or settings lowered quality         | Inspect top words (section 14); revert the change that caused it.                              |
| Out of disk space                                        | About 4 GB needed                             | Free space; the honeypot alone is about 2.5 GB including git objects.                          |

## 17. Metric glossary

| Term                | Meaning                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------|
| TP / FP / TN / FN   | Phishing caught / legitimate wrongly flagged / legitimate correctly passed / phishing missed     |
| Accuracy            | $(TP + TN) / \text{all}$ . Misleading when classes are imbalanced.                               |
| Precision           | $TP / (TP + FP)$ : how trustworthy a "phishing" verdict is                                       |
| Recall              | $TP / (TP + FN)$ : share of phishing caught                                                      |
| F1                  | Harmonic mean of precision and recall                                                            |
| False-positive rate | $FP / (FP + TN)$ : share of legitimate mail flagged; what annoys users                           |
| ROC-AUC             | Probability a random phishing email scores higher than a random legitimate one                   |
| PR-AUC              | Average precision across thresholds; informative for the minority (phishing) class               |
| Threshold           | Score above which an email counts as phishing: 0.5 for metrics, 0.8 for automatic quarantine     |
| Shortcut learning   | The model learns a dataset quirk (a name, a language, an era) instead of real phishing behaviour |
| Data leakage        | Test information reaching training (e.g. duplicates in both splits), inflating scores            |

## 18. Known limitations

- **No modern legitimate corpus exists publicly** (real inboxes are private). Legitimate training mail is from 2000-2008, so genuine modern "your account" notices can score high (3 of 28 legitimate modern-test messages flagged).
- **English only.** Non-English emails are not judged reliably by the text model; header and URL checks still apply to them.
- **Small modern test** (56 messages): those figures are indicative, not precise.
- Gift-card BEC, UPI and Hinglish scams are under-represented; some are still missed (see 13.3).
- The text model alone quarantines 67.4% of future phishing at the 0.8 threshold; the Day 6 scorer adds header and URL evidence.
- Hand-maintained lists (brands, shorteners, free hosts, risky TLDs) need periodic updates.

## 19. Licences and attribution

| Data                               | Licence / terms                                                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| phishing_pot                       | CC BY-NC 4.0 (verified from the repository's LICENSE file): attribution, <b>non-commercial use only</b>                                                                     |
| UCI SMS Spam Collection            | CC BY 4.0 as listed by UCI (not re-checked by the author: UCI was unreachable)                                                                                              |
| Kaggle compilation and its sources | Licence of the compilation <b>not verified</b> by the author (Kaggle was unreachable). Check the Kaggle page; each original corpus is a research corpus with its own terms. |
| GitHub mirror (rokibulroni)        | Unofficial copy without a licence file; the original sources' terms apply                                                                                                   |
| modern_lures.csv and all code      | This repository's licence (MIT)                                                                                                                                             |

This repository never redistributes the datasets; they are downloaded by each user and kept out of git. Cite the original authors when you publish results.